

Experiment 2

Fixed-point Pick-and-place with a mechArm 270

Laboratory Report

Course	Robotics Integration Group Project I
Group	Group 17
Members	Yang Muqing; Lan Songyu; Zhang Kairui; Zhao Hang
Platform	ROS 2 Humble + Gazebo + mechArm 270
Evidence version	experiment2_submission_final_20260913_v2
Repository	https://github.com/9161942-boop/robotics-experiment-2

This report documents a fixed-point manipulation experiment. The target is placed at a known point A, grasped with a parallel gripper, carried above a safe height, and released at point B. The report separates the control principle, the ROS 2 implementation, the verified simulation results, and the evidence available for the real-machine demonstration.

September 2026

Abstract

This laboratory experiment implements a fixed-point pick-and-place task for a six-joint mechArm 270 with a gripper. The system is organized as a ROS 2 workspace and uses a single Launch file to start the robot description, Gazebo scene, controllers, state publisher, visualization and task node. A predefined sequence moves the arm from home to the pick point A, closes the gripper, lifts the target cylinder, moves to the place point B, releases it through a short vertical drop, and returns home. The task node uses cosine interpolation between joint-space waypoints, publishes state and result topics, checks joint limits, and records every transition in CSV form. In the final v2 package, two independent five-cycle CSV logs both contain five `cycle_passed` records and a final `status: passed` summary with `success_count=5`. The supplied simulation and real-machine videos are included as visual evidence. The remaining quantitative limitation is that the real-machine video is not accompanied by a separate five-cycle numeric table; therefore this report distinguishes what is directly visible from what is supported by the simulation logs.

Keywords: ROS 2; Gazebo; mechArm 270; fixed-point manipulation; gripper control; laboratory report

1 Objective and Acceptance Requirements

1.1 Experiment objective

The objective is to build and document a complete manipulation pipeline rather than demonstrate an isolated joint command. The robot must be able to return to a known home configuration, move through a safe approach path, close and open the gripper, and transfer a target from a fixed pick point A to a fixed place point B. The visual-localization problem is outside this experiment: the points are configured in advance and the task is evaluated as a repeatable motion-and-control problem.

The intended workflow is first to verify the robot and controller in simulation and then to reuse the same ROS 2 interface on the physical arm. The real-machine change should be limited to device, communication and position parameters. This separation makes it possible to diagnose trajectory and state-machine errors without putting hardware at risk.

1.2 Acceptance checklist

Table 1. Requirement-to-evidence mapping

Requirement	Evidence in the v2 submission package
Complete ROS 2 program and one Launch file	<code>src/mecharm_fixed_pick_sim/</code> , <code>run_contact_grasp.sh</code> , Launch and parameter YAML files
Fixed A/B points, safe height and robot model	<code>fixed_point_experiment.yaml</code> , Gazebo world, URDF/Xacro and controller configurations
Five consecutive simulation grasps, at least four successes	Two independent CSV files; each has five <code>cycle_passed</code> rows and a final 5/5 passed summary
Collision/joint-limit and abnormal handling	Validation record, explicit joint-limit checks in the task node and <code>ERROR_LOG.md</code>
Simulation and real-machine demonstrations	<code>submission_evidence/videos/simulation_demo.webm</code> and <code>real_robot_demo.mp4</code>
Report and running instructions	This PDF/LaTeX report plus the root and package README files

Simulation acceptance status: the supplied final logs support 5 successful cycles out of 5. **Real-machine evidence status:** a demonstration video is supplied; a separate numeric five-cycle and safety-check record is not present in the archive and is therefore reported as supplementary evidence rather than inferred.

2 System Environment and Overall Workflow

2.1 Software and hardware environment

The workspace targets Ubuntu 22.04 with ROS 2 Humble and Gazebo Classic 11. The implementation is written in Python using `rc1py`. Its interfaces include joint-trajectory and joint-state messages, state/result strings, visualization markers and Gazebo entity-state services. The robot description is assembled from the `mechArm 270` URDF/Xacro files and the auxiliary `mycobot_description` package. Controllers are loaded for the arm, the main gripper, the contact pad and joint-state broadcasting.

2.2 Workflow

The experiment follows the sequence shown in Fig. 1. The source code and configuration are kept separate from generated logs, so a reviewer can reproduce the task without guessing which parameters were active during the final run.

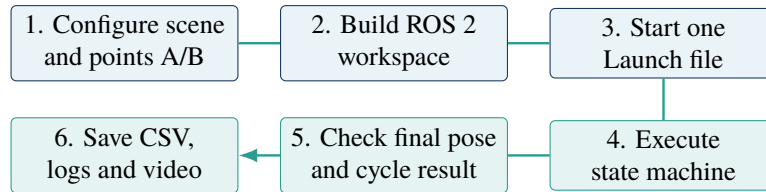


Figure 1. Experiment workflow from configuration to acceptance evidence.

3 Control Principle

3.1 Joint-space trajectory generation

The final configuration uses a set of manually selected joint vectors. For two consecutive waypoints $\mathbf{q}_0$ and $\mathbf{q}_1$, the task node uses a cosine interpolation parameter

$$s(t) = \frac{1 - \cos(\pi t/T)}{2}, \quad \mathbf{q}(t) = \mathbf{q}_0 + s(t)(\mathbf{q}_1 - \mathbf{q}_0),$$

where T is the step duration. The zero slope at the two ends reduces abrupt starts and stops. Commands are published at 30 Hz, while each waypoint transition lasts 1.6 s and is followed by a 0.35 s settling interval.

3.2 State machine and grasp semantics

The state machine is intentionally explicit: `initial`, `above_a`, `crab`, `pick`, `lift`, `initial_2`, `release_ready`, `release`, and the final `initial`. The gripper opens to 0.12 before approach and uses -0.04 for the final pick version. After the node confirms the grasp, the cylinder is visually attached to the gripper in the Gazebo workflow. At the release point, the node reads the actual target X/Y, moves to a position 0.02 m above that location, opens the gripper, and then fixes the target at $z = 0.020$ m. This vertical release avoids a lateral throw and makes the placement check reproducible.

3.3 Success and safety conditions

The final object pose is compared with B. A cycle passes when the object is below the configured maximum Z and the XY error is within 0.065 m. The task stops with a failure reason if a waypoint violates a joint limit, a configured fault mode reports an unreachable target or joint-limit fault, a service times out, or the final placement check fails. The final JSON summary records `success_count`, `repeat_count`, the success rule and the failure reason.

4 Scene and Parameter Configuration

Table 2. Final fixed-point experiment parameters

Parameter	Value
World frame	base
Table size	$0.60 \times 0.45 \times 0.04$ m
Pick point A	(0.30, 0.18, 0.031) m
Place point B	(0.2463, -0.1241, 0.020) m
Safe height	0.12 m
Target cylinder	radius 0.018 m, height 0.050 m
Placement tolerance	XY error ≤ 0.065 m; maximum Z 0.090 m
Control rate / step / settling	30 Hz / 1.6 s / 0.35 s
Repeat count	5
Gripper open / pick	0.12 / -0.04

The green placement area is centered on B. The target cylinder and the support geometry were tuned to radius 0.018 m and 0.020 m, respectively, while the joint trajectory itself was kept unchanged. This is a useful engineering distinction: the geometry was adjusted to match the contact envelope and release area, whereas the taught motion was not silently changed.

5 ROS 2 Implementation and Operation

5.1 Package organization

The package `mecharm_fixed_pick_sim` contains the task node, Launch files, configuration, world and robot description. `mycobot_description` provides meshes and gripper resources. The root script starts the complete experiment and writes logs into `logs/fixed_pick/`. The submission evidence is intentionally outside the source package so that code and results can be reviewed independently.

5.2 Command-level procedure

The following commands describe the actual operation record included in the README. They are written as a reproducible sequence rather than as pseudocode.

1. Source ROS 2 and build the workspace:

```
source /opt/ros/humble/setup.bash
colcon build --symlink-install
source install/local_setup.bash
```

2. Start the final five-cycle task with the provided script:

```
./run_contact_grasp.sh
```

The script checks for existing Gazebo processes, clears stale overlay variables, sets the Gazebo master URI, and passes `repeat_count:=5`, `exit_on_complete:=true` and a startup delay to the Launch file.

3. For manual launch, the equivalent ROS 2 command is:

```
ros2 launch mecharm_fixed_pick_sim fixed_point_experiment.launch.py \\  
  contact_grasp:=true gui:=true run_task:=true repeat_count:=5 \\  
  exit_on_complete:=true task_start_delay_sec:=20.0
```

4. Inspect the CSV for `grasp_attached`, `released`, `release_settled`, `cycle_passed` and the final JSON summary. The node's topic-level status and the final object pose are both checked; a controller message alone is not treated as proof of placement.

5.3 ROS 2 interfaces

The node publishes a state string for the current phase, a result string for terminal status, joint commands for the arm and gripper, and visualization markers for the configured points. The Launch file exposes parameters for GUI mode, task start, repeat count, exit behavior, startup delay and log directory. These parameters allow simulation and hardware runs to share the same control logic while changing only the environment-specific values.

5.4 Simulation implementation process

The simulation followed seven explicit steps: manual teaching and point adjustment came first, followed by automatic playback and acceptance checking.

1. **Build the Gazebo scene.** A `mecharm_fixed_pick_sim` workspace was prepared under ROS 2 Humble and Gazebo Classic 11. The scene loaded the `mechArm 270`, parallel gripper, tabletop, target cylinder and placement area. Mass, inertia, collision bodies and friction coefficients were configured for the target so that contact, gripping, lifting and gravity release produced observable physical responses.
2. **Compile and load the robot model.** The workspace was built with `colcon build --symlink-install` and the local overlay was sourced. The URDF/Xacro files, joint names, gripper joint and controller configuration were checked so that the manual control panel and automatic task node used the same joint interfaces.
3. **Start Gazebo and teach poses manually.** One terminal launched Gazebo and the controllers with `ros2 launch mecharm_fixed_pick_sim official_assembled_gazebo.launch.py gui:=true run_task:=false contact_grasp:=true`. A second terminal ran `ros2 run mecharm_fixed_pick_sim manual_control_panel`. The panel provided six joint sliders, a gripper slider and a contact-pad force slider. Clicking “Send current pose” sent the selected joint state to Gazebo. The arm was moved step by step to the approach, pick, lift, transfer and release poses while the end-effector clearance and gripper orientation were observed. “Reset object to A” restarted a debugging trial.
4. **Record waypoints and tune the parameters.** The panel recorded six joint angles, gripper position, contact-pad force and motion duration for each waypoint. The points `above_pick`, `down_pick`, `grasp_object`, `lift_object`, `move_to_place` and `release_object` were added in execution order and saved to `manual_animation.json`;

the previous file was retained as `manual_animation.json.bak`. Manual short cycles were used for parameter tuning: collision bodies, mass, inertia, friction and contact force were adjusted when the object was ejected or unstable; contact geometry and speed were adjusted when penetration or vibration occurred; B coordinates, release height, motion duration and gripper opening time were adjusted when the placement was offset.

5. **Launch the automatic simulation.** After the manually taught poses were stable, `run_contact_grasp.sh` launched the five-cycle task. The script cleaned stale Gazebo processes and environment variables, selected a dedicated Gazebo master URI, waited for controller initialization and passed the confirmed action sequence to the task node.
6. **Replay the sequence and check acceptance.** The automatic task node read the waypoints in `manual_animation.json` and executed the `initial`, `above_a`, `crab`, `pick`, `lift`, `release_ready`, `release` and `reset` phases. Cosine interpolation generated 30 Hz joint commands. The program completed five cycles, checked the final X/Y/Z, XY error, height, joint limits, service responses and state transitions, and wrote the cycle number, phase, joint positions, gripper actions and checks to CSV. The terminal summary, two five-cycle CSV files, error log, checksums and simulation video were archived together.

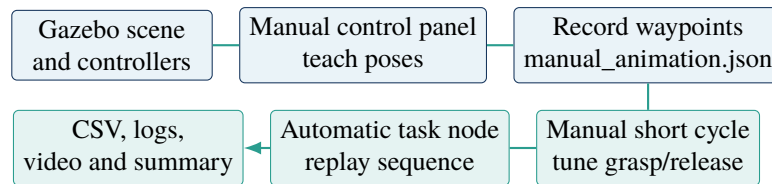


Figure 2. Simulation flow from manual teaching to automatic replay and evidence generation.

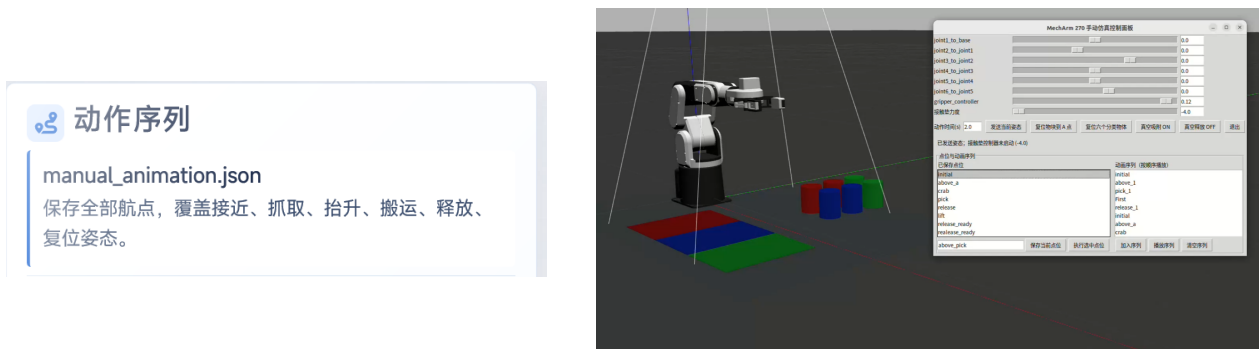


Figure 3. Simulation debugging artifacts: the waypoint sequence and the actual Gazebo manual control console.

5.5 Real-machine implementation process

The real-machine demonstration followed six steps that corresponded to the simulation phases.

1. **Prepare the platform and communication.** The setup used a six-DOF mechArm 270-Pi with an electric gripper. `pymycobot` connected to `/dev/ttyAMA0` at 1000000 baud. Power, serial communication, emergency stop, controller connection and gripper opening/closing were checked before motion.
2. **Arrange the tabletop and A/B points.** The arm was placed beside the prepared tabletop. The fixed pick point A and place point B were arranged with the same relative geometry used in Gazebo. Base clearance,

table edges and the placement area were checked so that SAFE, PICK and PLACE poses approached the table vertically with sufficient clearance.

3. **Set HOME and perform a low-speed check.** The arm was returned to HOME with the gripper open. A no-load, low-speed cycle checked joint directions, table clearance and gripper orientation. This cycle confirmed the physical path before the target was introduced.
4. **Execute the fixed-point grasp.** The program moved from HOME to SAFE above A, descended to PICK, closed the gripper at GRIP and lifted back to SAFE. The operator observed whether the target stayed centered in the gripper and whether lifting caused slip or contact.
5. **Transfer, release and reset.** The arm moved along the taught path to SAFE above B, descended to PLACE, opened at RELEASE, returned to SAFE and moved back to HOME. Target position, end-effector height and return posture were checked as placement, clearance and reset conditions.
6. **Record evidence and handle abnormal states.** The demonstration was observed for grip stability, table contact, placement offset and serial status. A communication fault, target slip or abnormal posture triggered a stop of continuous motion and a return to the safe posture. The supplied MP4 records approach, grasp, lift, transfer, release and reset; the simulation CSV supplies the five-cycle numerical result.

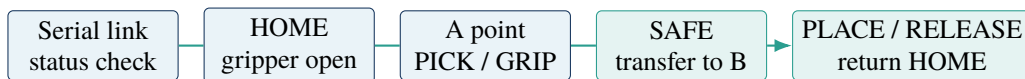


Figure 4. Real-machine action chain from device preparation to grasp, transfer, release and reset.

6 Verification Results

6.1 Five-cycle simulation record

The two CSV files in the v2 package are distinct SHA-256-verified records. Each contains 116 rows: five complete cycles followed by a terminal summary row. The five `cycle_passed` rows in the recorded run have the following final positions and XY errors.

Table 3. Five-cycle result from `fixed_pick_20260912_233915.csv`

Cycle	x (m)	y (m)	z (m)	XY error (m)
1	0.2457	-0.1238	0.0200	0.0006
2	0.2462	-0.1241	0.0200	0.0001
3	0.2459	-0.1239	0.0200	0.0005
4	0.2466	-0.1242	0.0200	0.0003
5	0.2459	-0.1239	0.0200	0.0005
Maximum XY error				0.0006

The terminal row reports `success_count=5`, `repeat_count=5` and the rule “at least 4 successful runs out of 5.” The second CSV gives the same 5/5 result with a maximum XY error of 0.0005 m. The extra repeat value 6 in the CSV is the final summary row emitted after cycle 5; it is not an additional grasp attempt.

6.2 Static and abnormal-condition checks

The validation record reports successful Python syntax, Xacro, URDF and `colcon build` checks. Before motion, the task node validates each waypoint against the configured joint limits. The error log records the engineering issues encountered while converging to the final version: a simplified gripper collision model could eject the target, an oversized contact envelope could cause penetration, an insufficient startup delay could race controller activation, and a reused release waypoint could create overlapping placements. These cases were resolved by using the contact-mode model, reducing the contact geometry, delaying task start until controllers were ready, and checking the final Gazebo pose after release.

7 Demonstration Evidence

7.1 Simulation video

Figure 5 is a three-frame summary extracted from `simulation_demo.webm`. The first frame shows the arm approaching the working area, the middle frame shows the gripper carrying the red cylinder, and the last frame shows the robot back at its initial side of the scene after release. The original WebM is included unchanged in the submission package.

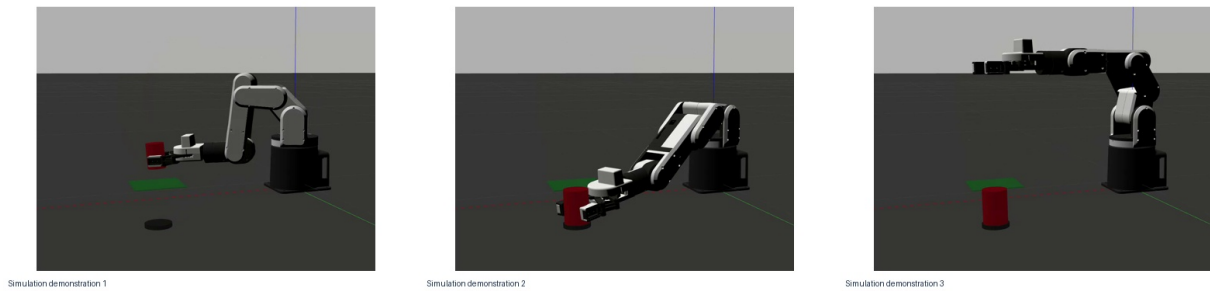


Figure 5. Selected frames from the supplied simulation demonstration video.

7.2 Real-machine video

Figure 6 summarizes the supplied portrait MP4. It clearly shows the physical arm, gripper, table and target area during operation. It is appropriate evidence that the motion was transferred to hardware. Because the video is not accompanied by a timestamped five-cycle table or a separate safety checklist, the report does not convert the video alone into a numeric success rate.

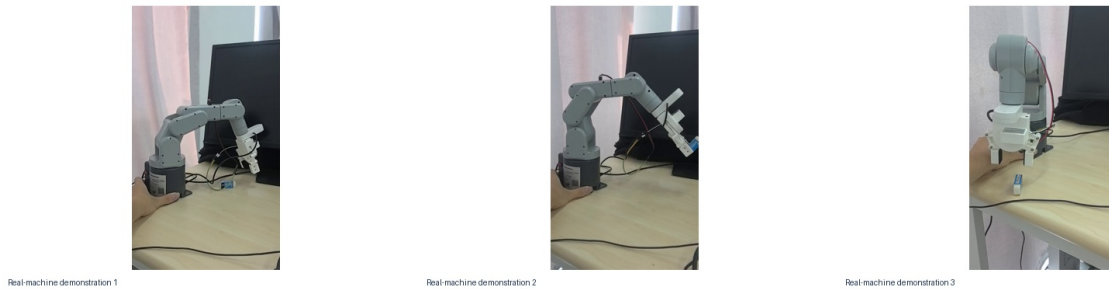


Figure 6. Selected frames from the supplied real-machine demonstration video.

8 Discussion and Summary

8.1 What the experiment demonstrates

The final simulation package demonstrates a complete fixed-point manipulation loop with explicit parameters, a reproducible state machine, controller startup, object attachment and release semantics, final-pose checking and persistent CSV output. The five-cycle records are stronger than a single successful screenshot because they show repeatability and expose the actual release coordinates. The small maximum XY errors also show that the final vertical-release strategy is consistent with the configured B point.

The source organization makes the experiment easier to reproduce. A new user can build the workspace, run one script, inspect the same named phases, and find the corresponding rows in the log. The separation between source, configuration, evidence and report also supports staged GitHub commits and makes it possible to review changes without downloading a binary archive.

8.2 Limitations and practical lessons

The fixed-point task assumes that A and B are known in advance, so it does not solve camera-based localization or dynamic obstacle avoidance. The Gazebo visual attachment workflow is deliberately controlled to make grasp and release measurable; a physical gripper has additional effects such as compliance, friction variation, cable interference and object slip. The supplied real-machine video therefore complements, but does not replace, a numeric hardware test record.

The debugging history shows why final-pose verification matters. A node can report that a gripper closed or that an object was visually attached while the physical or simulated object still ends outside the target area. Checking the object's final X/Y/Z, recording the release-above and release-settled events, and preserving the error log turns a visually plausible demo into a traceable experiment.

8.3 Summary and reflection

1. A fixed-point manipulation task is primarily a consistency problem: coordinate frames, taught joint vectors, gripper values, object geometry and success thresholds must describe the same scene.
2. A state machine with explicit phases is easier to diagnose than a long unlabelled command stream. The phase names in the CSV make it possible to locate the first failed action.
3. Safe operation requires checks before motion and after release. Joint-limit validation prevents invalid commands, while final-pose checking prevents false positives.
4. ROS 2 Launch and parameter files provide the bridge from simulation to hardware. The control sequence can remain stable while device and environment parameters change.
5. The final deliverable is more useful when it contains source, logs, videos, report source and PDF together, with checksums for the evidence files and a staged Git history that records how the package was assembled.

9 Submission File Map

Table 4. Files included in the final submission directory

Path	Purpose
src/mecharm_fixed_pick_sim/	ROS 2 task node, Launch, YAML, world and URDF/Xacro
src/mycobot_description/	Robot meshes and gripper description resources
run_contact_grasp.sh	One-command five-cycle simulation launch
submission_evidence/results/	Two independent five-cycle CSV records
submission_evidence/docs/ERROR_LOG.md	Abnormal cases and fixes
submission_evidence/videos/	Simulation WebM and real-machine MP4
submission_evidence/SUMS.txt	Hashes for the evidence files
report/experiment2_lab_report.tex	This report source
report/experiment2_lab_report.pdf	Compiled report for submission

9.1 GitHub commit record

The experiment-two materials are maintained in the dedicated public repository <https://github.com/9161942-boop/robotics-experiment-2>. The upload was split into four commits so that the code, evidence, report and videos can be reviewed independently:

Table 5. Staged GitHub commits for Experiment 2

Commit	Content
b21bb71	ROS 2 packages, Launch files and simulation configuration
330ac6a	Five-cycle CSV records, error log and checksums
dc85c53	English LaTeX report, PDF and figures
b4f67d2	Simulation and real-machine demonstration videos

Declaration of Evidence Scope

The simulation results in this report are quoted from the supplied final v2 archive and its CSV/validation records; no new simulation was run while preparing this report. The two videos are the user-supplied demonstration files and were inspected read-only. Statements about the exact five-cycle simulation count are supported by the CSV files. Statements about the real-machine demonstration are limited to what is visible in the supplied MP4 unless a separate hardware result table is added later.

Figure 4. GitHub commit history for the public Experiment 2 repository

